Edge-Compute Feasibility of Cooperative Perception Fusion for Connected Autonomous Vehicles

Comparing V2VNet and when2com's onboard compute cost against real edge hardware budgets, using hands-on profiling rather than published numbers alone

Ganesh Adhikari | CS 7000 | **Revised Oct 3: addresses advisor's second round of feedback (20+ scene dataset, visual detection renderings, per-class AP)**

Background

Vehicle-to-everything (V2X) communication allows connected vehicles and roadside units to share sensor data with each other, helping vehicles see through occlusion and beyond the range of their own onboard sensors. Fusion methods such as V2VNet and when2com combine this shared data into a single scene understanding. For this to work in a real vehicle, the fusion model itself must eventually run on limited onboard hardware, not just transmit data efficiently over the network. Existing cooperative perception research, including the Where2comm line of work, extensively studies the trade-off between detection accuracy and communication bandwidth, meaning how many bytes are transmitted between vehicles. Far less attention has been given to the onboard model compute cost itself, meaning whether the fusion model is small and fast enough to run within a real edge device's memory and processing budget.

This is the core reason for the “feasibility” framing in the title: the project measures actual, hands-on compute profiles of the two methods, rather than only comparing numbers already published in the original papers, so that the feasibility claim is backed by direct measurement.

Research Question

Between V2X-Sim's existing V2VNet and when2com baselines (existing, already-published methods), which offers the best detection improvement per unit of on-device compute cost (which method gives the most benefit for the least computing effort), and would either realistically fit a Jetson Nano-class edge board's memory and processing budget?

Method

The project follows a single, sequential pipeline:

- Set up the V2X-Sim 2.0 dataset and the coperception codebase, which includes pretrained checkpoints for the single-vehicle (lowerbound), V2VNet, and when2com methods. **[UPDATED]** Scaled from the original 1-scene mini dataset (~6 GB) to a 20-scene subset (~14 GB preprocessed), with a genuine 14/3/3 train/validation/test split, giving a real held-out test set rather than evaluating on training scenes alone.
- Read the original V2VNet (arXiv 2008.07519, ECCV 2020) and when2com (arXiv 2006.00176, CVPR 2020) papers before implementation, as directed by the advisor, to confirm the fusion mechanisms and evaluation protocol are understood correctly.
- Run each pretrained model to confirm baseline detection accuracy (mean Average Precision) matches previously published numbers, computed on the full 20-scene test split.
- Profile each model directly, rather than citing paper-reported numbers: parameter count, model size in megabytes, per-layer latency breakdown, peak memory usage during inference, and end-to-end inference latency, measured over repeated runs on available hardware (a laptop or a free-tier cloud GPU such as Google Colab).
- **[NEW]** Render qualitative detection results as point-cloud and image visualizations for a sample of frames, alongside the quantitative mAP tables, so results can be inspected visually rather than as numbers alone.
- Calculate an efficiency ratio for each method: the detection accuracy gain over the single-vehicle baseline, divided by model size and by inference time.
- Compare these measured profiles against the Jetson Nano's published specifications (4 GB memory, 472 GFLOPS compute) to produce a paper-based feasibility estimate, since no physical Jetson Nano hardware is required or used.
- Translate the feasibility verdict into a concrete recommendation: if neither model fits the Jetson Nano's practical budget as-is, propose a specific next step (e.g., which method is the better starting point, and which optimization — pruning, quantization, or reduced feature resolution — would most plausibly close the gap), rather than stopping at a pass/fail comparison.

Metrics

- Detection mAP for all three methods (single-vehicle baseline, V2VNet, when2com), at IoU 0.5 and 0.7
- **[NEW]** Per-class AP (car, bus, bicycle/motorcycle), in addition to overall mAP, to show whether accuracy gains are uniform across object types or concentrated in specific classes
- Model size (MB) and parameter count
- Peak memory usage (MB) during inference, measured directly rather than estimated

- Estimated inference latency (roughly how long the model takes to process one frame and produce a result)
- Per-layer / per-module latency breakdown to identify which components of each model drive the compute cost
- Accuracy gain per MB and per millisecond (efficiency ratio)
- Feasibility verdict against Jetson Nano's published specifications
- **[NEW]** Point-cloud and image renderings of detections for a sample of frames per method, used as qualitative, visual evidence alongside the numeric metrics

Expected Outcomes

- A clear comparison of which fusion method is more efficient for edge deployment, not just more accurate.
- A feasibility verdict on whether either method realistically fits Jetson Nano-class hardware constraints.
- A specific, actionable recommendation for practitioners — which method to start from and what optimization to apply — rather than metrics alone, directly addressing the advisor's request for a solution/suggestion.
- **[NEW]** Visual, per-class evidence (detection renderings and per-class AP) supporting the quantitative mAP comparison, rather than relying on a single aggregate number.
- Practical guidance for choosing a cooperative perception method under real edge compute constraints.

Hypothesis

When2com's lightweight, attention-based communication mechanism will offer a better accuracy-per-compute trade-off than V2VNet's denser fusion approach, though both may exceed the Jetson Nano's practical real-time budget without further optimization. Direct profiling is expected to show that the compute bottleneck is concentrated in specific layers (e.g., the fusion/aggregation module rather than the backbone), which would itself point toward a targeted optimization recommendation.

Scope Note on Hardware Feasibility

This project does not run models on physical Jetson Nano hardware. Feasibility is assessed by comparing directly measured model profiles (size, memory, latency) against the device's published specifications — an estimate rather than a hardware benchmark. Real device performance can vary due to memory bandwidth, thermal throttling, and software optimization differences. This limitation is stated explicitly in the paper rather than presented as a hardware-validated result.

Why This Is Defendable

- Prior work optimizes communication bandwidth (bytes transmitted), not onboard inference compute cost measured against a specific real edge device's published specifications.
- Direct, hands-on profiling (rather than citing accuracy/size numbers from the original papers) makes this a measurement-driven feasibility study, addressing the concern that the original framing leaned too heavily on comparison alone.
- **[UPDATED]** A 20-scene dataset with a genuine train/validation/test split, rather than the original 1-scene mini dataset, gives the mAP and per-class AP results real held-out evidence instead of numbers drawn from training scenes.
- Both baselines already exist with pretrained checkpoints in the public coperception codebase, making this a measurement and feasibility study rather than new model development, appropriate in scope for a one-semester project.
- Lower implementation risk than training a new model from scratch, since evaluation can begin directly from existing pretrained checkpoints.

12-Week Timeline

Week	Task
1	Set up V2X-Sim dataset, coperception codebase, and pretrained checkpoints Read the V2VNet and when2com papers in full before implementation
2–3	Reproduce baseline accuracy (mAP) numbers for all three checkpoints; draft Background and Related Work sections **[UPDATED]** Expand dataset from 1-scene mini to 20-scene (14/3/3 split) for genuine held-out evaluation
4–5	Build the compute profiling pipeline: model size, parameter count, latency, peak memory usage, and per-layer latency breakdown, run over repeated trials Draft Method section
6–7	Run full profiling across all three models on the 20-scene test split; compute efficiency ratios and per-class AP **[NEW]** Generate point-cloud/image renderings of detections for qualitative results
8	Compare against Jetson Nano specs; produce feasibility verdict; draft Results section with visualizations and per-class AP tables
9	Interpret findings and finalize a concrete recommendation (method + optimization path) Draft Discussion section
10	Assemble full draft; generate all figures and tables

Week	Task
11	Advisor feedback and revisions
12	Final edits and submission